

Feature Alignment & Cross-Modal Attention.

A multimodal LLM (LLaVA-style) bolts a frozen vision encoder onto a language model. The trick is almost embarrassingly simple: a small learned *projection* maps each visual patch feature into the LLM’s token-embedding space, turning the image into a handful of “visual tokens” that sit in the same sequence as text. From there the LLM’s *ordinary* self-attention is already cross-modal — no new operator is needed. Below we derive the projection, count its parameters, and compute a full cross-modal attention row by hand. Every number on these pages is computed in full — nothing is summarised or deferred.

THE EQUATION

$$\underbrace{H_v = \phi(F_v) \in \mathbb{R}^{N_v \times d_{\text{model}}}}_{\text{align}}, \quad \text{Attn}\left(\underbrace{[H_v; H_t]}_{\text{one sequence}}\right) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right) V$$

Worked example. A ViT giving $N_v = 576$ patches of $d_v = 1024$, projected to $d_{\text{model}} = 4096$; the projector’s parameter count; and a full softmax attention row for one text query over a mixed 3+2 token sequence, shown as a two-tone heatmap.

Topic 1 of 1 in this module · companion to the course page · v1.0

Contents

1	The claim: align, then attend	2
2	Architecture diagram	3
3	(a) Feature alignment: the projector ϕ	3
4	(b) Cross-modal attention falls out for free	4
5	Two ways to wire it up	5
6	The cost of longer sequences	5
7	Where this sits in the track	6
A	Reproducible (NumPy)	6

1 The claim: align, then attend

A vision encoder (a ViT) turns an image into N_v patch features of dimension d_v . An LLM consumes token embeddings of dimension d_{model} . These two spaces are unrelated: $d_v \neq d_{\text{model}}$, and even where the dimensions matched the geometry would not. The multimodal recipe is two steps.

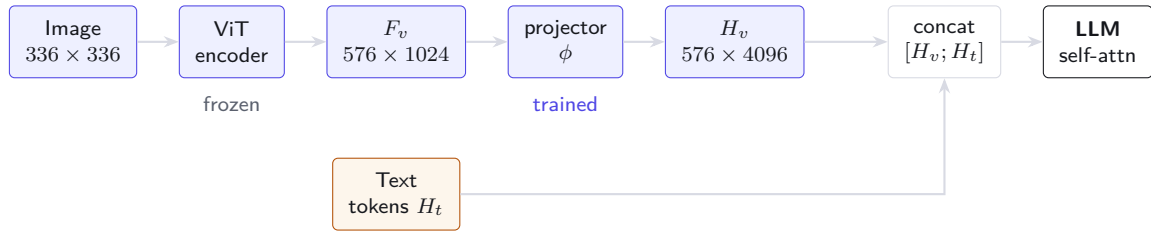
$$\underbrace{H_v = \phi(F_v) \in \mathbb{R}^{N_v \times d_{\text{model}}}}_{\text{(a) feature alignment}} \quad \underbrace{\text{Attention}([H_v; H_t]) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right)V}_{\text{(b) cross-modal attention}} \quad (1)$$

Here $F_v \in \mathbb{R}^{N_v \times d_v}$ are the raw ViT features, ϕ is a small learned *connector* (the projector), $H_t \in \mathbb{R}^{N_t \times d_{\text{model}}}$ are the text-token embeddings, and $[\cdot; \cdot]$ stacks rows into one length- $(N_v + N_t)$ sequence. The mask M is the LLM's usual causal mask, simply extended over the longer sequence.

Symbol	Shape	Meaning
F_v	$N_v \times d_v$	ViT patch features (output of vision encoder)
ϕ (projector)	$d_v \rightarrow d_{\text{model}}$	learned MLP “connector”
$H_v = \phi(F_v)$	$N_v \times d_{\text{model}}$	<i>visual tokens</i> in the LLM’s space
H_t	$N_t \times d_{\text{model}}$	text-token embeddings
$X = [H_v; H_t]$	$(N_v + N_t) \times d_{\text{model}}$	the single fused sequence
$Q, K, V = XW_{q,k,v}$	$(N_v + N_t) \times d$	queries / keys / values (one head)
M	$(N_v + N_t)^2$	causal mask ($-\infty$ above diagonal)

The image is translated into the LLM’s own “language”: a short paragraph of N_v extra tokens that happen to encode pixels instead of words. Once they live in the embedding space, the LLM cannot tell — and does not care — whether a token came from an eye or a keyboard. Attention treats them all alike.

2 Architecture diagram



Only the small projector (and, in a later stage, the LLM) is trained; the heavy vision encoder stays frozen. The concatenation block is the entire “fusion” — no special cross-modal layer.

3 (a) Feature alignment: the projector ϕ

A ViT with a 14×14 patch size applied to a 336×336 image produces a $\frac{336}{14} \times \frac{336}{14} = 24 \times 24$ grid of patches, hence

$$N_v = 24 \times 24 = 576 \text{ patch features,} \quad \text{each of dimension } d_v = 1024.$$

The LLM works in $d_{\text{model}} = 4096$. The simplest connector is a single linear map $W \in \mathbb{R}^{d_v \times d_{\text{model}}}$ applied to every patch row:

$$H_v = F_v W, \quad F_v \in \mathbb{R}^{576 \times 1024}, \quad W \in \mathbb{R}^{1024 \times 4096}, \quad H_v \in \mathbb{R}^{576 \times 4096} \quad (2)$$

Each of the 576 rows is mapped *independently* (the same W for every patch), turning 576 vision features into 576 visual tokens the LLM can read. The parameter count is

$$\#W = d_v \times d_{\text{model}} = 1024 \times 4096 = 4,194,304 \approx 4.19\text{M}.$$

LLaVA-1.5 uses a two-layer MLP $d_v \rightarrow d_{\text{model}} \rightarrow d_{\text{model}}$ with a GELU in between, which has

$$\underbrace{1024 \times 4096}_{4.19\text{M}} + \underbrace{4096 \times 4096}_{16.78\text{M}} = 20,971,520 \approx 21.0\text{M parameters}$$

— still tiny next to a 7B-parameter LLM ($\sim 0.3\%$). That is the whole point: alignment is cheap.

“Alignment” is just a learned change of basis. The projector does not extract new information from the image; it *re-coordinates* the ViT’s features so they land where the LLM expects its word embeddings to live. A 4.19M matrix buys the LLM a pair of eyes.

4 (b) Cross-modal attention falls out for free

Once the visual tokens share the LLM’s space, fuse the two streams into one sequence

$$X = [H_v; H_t] \in \mathbb{R}^{(N_v + N_t) \times d_{\text{model}}},$$

project to Q, K, V , and run the *identical* attention operator from Module 0.1:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right)V.$$

A text query at position i produces score row $S_{i,:} = q_i K^\top / \sqrt{d}$ over *all* keys — visual and textual alike. There is no separate “image attention”; the same dot products that let a text token attend to earlier text tokens let it attend to the visual tokens. The causal mask M is the only bookkeeping: visual tokens are placed first, so every text token sees the whole image, exactly as it sees all preceding words.

4.1 A mixed sequence, multiplied out

Take a tiny example: $N_v = 3$ visual tokens v_1, v_2, v_3 followed by $N_t = 2$ text tokens t_1, t_2 , with head dimension $d = 4$. Using identity projections ($Q = K = X$) so the arithmetic is transparent, the rows are

$$X = \begin{bmatrix} 1.0 & 0.5 & -0.5 & 0.2 \\ 0.3 & -0.4 & 0.9 & 0.1 \\ -0.2 & 0.7 & 0.4 & -0.6 \\ 0.6 & 0.1 & -0.3 & 0.8 \\ 0.4 & -0.2 & 0.5 & 0.3 \end{bmatrix} \begin{matrix} v_1 \\ v_2 \\ v_3 \\ t_1 \\ t_2 \end{matrix} \quad q = q_{t_2} = [0.4 \quad -0.2 \quad 0.5 \quad 0.3].$$

The last text token t_2 is at the final position, so its causal mask lets it attend to all five tokens. Its raw dot products with each key $K_j \cdot q$ are

$$Kq^\top = [0.11 \quad 0.68 \quad -0.20 \quad 0.31 \quad 0.54]^\top,$$

e.g. for v_2 : $0.3(0.4) + (-0.4)(-0.2) + 0.9(0.5) + 0.1(0.3) = 0.12 + 0.08 + 0.45 + 0.03 = 0.68$. Scale by $\sqrt{d} = 2$:

$$S_{t_2,:} = \frac{1}{2} Kq^\top = [0.055 \quad 0.340 \quad -0.100 \quad 0.155 \quad 0.270].$$

Row-wise softmax gives the cross-modal attention weights:

$$p = \text{softmax}(S_{t_2,:}) = [0.181 \quad 0.240 \quad 0.155 \quad 0.200 \quad 0.224], \quad \sum_j p_j = 1.$$

4.2 The heatmap (visual vs. text)

The same row as a heatmap. **Indigo** cells are *visual* tokens, **amber** cells are *text* tokens — one attention distribution spanning both modalities:

	v_1	v_2	v_3	t_1	t_2
q_{t_2}	0.181	0.240	0.155	0.200	0.224

The query splits its mass 0.576 onto the three visual tokens and 0.424 onto the two text tokens: the text token is genuinely reading the image. The darkest cell is the visual token v_2 , the patch most aligned with what t_2 is asking about.

Nothing in the softmax knows that v_1, v_2, v_3 are “pixels” and t_1, t_2 are “words.” The weights 0.181, ..., 0.224 are computed by one formula over one sequence. Cross-modal fusion is not a new mechanism — it is the *absence* of a boundary.

5 Two ways to wire it up

Design	How vision enters	Example
Project + concatenate	visual tokens prepended to text; shared self-attn	LLaVA
Gated cross-attention	new cross-attn layers; text <i>queries</i> , vision <i>keys/values</i>	Flamingo

(i) **Projection + concatenate** (LLaVA) needs no new layers — only the projector — but pays for the image with extra sequence length in *every* attention layer. (ii) **Gated cross-attention** (Flamingo) inserts dedicated cross-attention blocks in which text tokens attend to a small fixed set of vision features, gated by a learned tanh scalar initialised at 0 so the pretrained LLM is unchanged at the start of training. It keeps the text sequence short but adds parameters and new layers.

Training stages (LLaVA recipe). (1) *Freeze* the vision encoder *and* the LLM; train only the projector ϕ on image–caption pairs so the visual tokens land in the right space. (2) *Unfreeze* the LLM and finetune it together with ϕ on instruction data. Staging keeps the expensive LLM stable while the cheap connector finds the alignment.

6 The cost of longer sequences

The convenience of design (i) has a price. Attention is $O(T^2)$ in the sequence length (Module 5). Prepending $N_v = 576$ visual tokens inflates T , and the cost grows with the *square* of the inflated length. For a text prompt of N_t tokens:

Text N_t	+ N_v visual	Total T	Attn cost vs. text-only
128	+576	704	30.3×
512	+576	1,088	4.52×
1,024	+576	1,600	2.44×

The multiplier is $(T/N_t)^2$: at a short 128-token prompt the image makes attention **30×** more expensive. High-resolution variants make it worse — LLaVA-1.6 tiles the image into up to 5 crops, i.e. $5 \times 576 = 2,880$ visual tokens before a word is even typed.

Visual tokens are not free real estate. Each one is a full position in the sequence, so it joins *every* pairwise dot product in *every* layer. Doubling resolution roughly quadruples N_v and can $16\times$ the attention cost (Module 5's $O(T^2)$). Token-reduction tricks — pooling patches, perceiver resamplers, or Flamingo-style fixed-size cross-attention — exist precisely to stop the image from dominating the budget.

7 Where this sits in the track

Link	What it gives	Module
Scaled dot-product attention	the operator reused, unchanged, for fusion	0.1
Causal masking	the M that extends over visual tokens	0.1
$O(T^2)$ bottleneck	why +576 visual tokens hurt	5.1

A multimodal LLM is mostly old parts: a ViT, an LLM, and the Module 0.1 attention you already know. The new content is a ~ 4 –21M-parameter projector and the discipline to stage the training. The cross-modal magic is that there is no magic.

A Reproducible (NumPy)

Inputs: $N_v = 576$, $d_v = 1024$, $d_{\text{model}} = 4096$ for the projector; seed-free fixed matrix X (3 visual + 2 text rows), $d = 4$, for the attention row.

```
import numpy as np
# --- (a) projector params ---
d_v, d_model, N_v = 1024, 4096, 576
print(N_v, d_v*d_model)           # 576 4194304 (~4.19M, single linear)
print(d_v*d_model + d_model*d_model) # 20971520 (~21.0M, 2-layer MLP)

# --- (b) cross-modal attention row for text query t2 ---
d = 4
X = np.array([[ 1.0, 0.5,-0.5, 0.2],      # v1
               [ 0.3,-0.4, 0.9, 0.1],      # v2
               [-0.2, 0.7, 0.4,-0.6],      # v3
               [ 0.6, 0.1,-0.3, 0.8],      # t1
               [ 0.4,-0.2, 0.5, 0.3]])      # t2 (query)
```

```
Q = K = X                                # identity projections
q = Q[4]                                  # t2 attends to all 5
S = (K @ q) / np.sqrt(d)
p = np.exp(S - S.max()); p /= p.sum()
print(np.round(K @ q, 3))                 # [ 0.11 0.68 -0.2  0.31 0.54]
print(np.round(S, 3))                     # [ 0.055 0.34 -0.1  0.155 0.27]
print(np.round(p, 3))                     # [ 0.181 0.24 0.155 0.2 0.224]
print(round(p[:3].sum(),3), round(p[3:].sum(),3)) # 0.576 0.424

# --- sequence inflation cost vs text-only ---
for nt in [128, 512, 1024]:
    T = nt + N_v
    print(nt, T, round((T/nt)**2, 2))      # 30.25 / 4.52 / 2.44
```